Ariadne

A structural type-theoretic reconstruction

Immutable evidence, local graphs, possible origins,
bounded frame questions and justified claims

Source snapshot

`184a181a0a09c81bed38c543721bd10970512d89`

Reading date: 20 October 2026

Delivery: LaTeX source and PDF; explanation only

Ariadne takes one immutable captured snapshot, constructs a finite local control-flow problem, and computes possible value origins. Its investigation layers attach questions and evidence to that problem. The useful type-theoretic view is a family of structured data types refined by validity, coverage and premise conditions.

This document reconstructs the current implementation in mathematical notation. It is not a new checked type theory, an implementation plan, or a replacement for the authoritative analysis-level TLA+ specifications. Rust and OCaml do not implement the dependent types written here.

Reading route

Sections 1–4 connect the explanation to concrete code. Sections 5–10 reconstruct the analysis machine. Sections 11–16 give types, judgments and transition rules. Sections 17–19 map them back to source and explain their limits. The appendices expand the frame, resolver and completion contracts.

Contents

1 Purpose and module boundary **3**

2 Concrete implementation **3**

3 Concrete invariants and their enforcement **4**

4 Representative execution in the code **5**

5 First abstraction boundary **5**

6 The minimal abstract machine **6**

7 Abstract operations **6**

8 Reachability and invariants **7**

9 Abstract execution trace **7**

10 Second abstraction boundary **7**

11 Abstract syntax and structural types **8**

11.1 The notation and its limits 8

11.2 Identity and instruction structure 8

11.3 A request is a refined record 8

12 Bindings and scopes **9**

13 Judgment forms **9**

14 Statics: validity and permission to make a claim **10**

14.1 Request well-formedness 10

14.2 Admission has more premises than shape 10

15 Dynamics: recovery, fixed point and slice **10**

15.1 Selected bytes and recovered control 10

15.2 Reaching definitions are a monotone finite problem 11

15.3 Phase boundaries and the backward closure 12

16 Errors, unknowns and partial results **12**

17 Implementation correspondence **12**

18 Final abstraction boundary **13**

19 Model versus implementation: what the types do not prove **14**
19.1 Excess representable states . . . 14
19.2 Preservation and progress, with their boundaries . . . 14
19.3 Which questions are computable here? . . . 14

A Frame products as a family of structural interfaces **15**
A.1 Stage 5.1: publish structure before full dataflow . . . 15
A.2 Stage 5.2: corridor, SCCs and bounded examples . . . 15
A.3 Full frame paths are a separate product . . . 16

B Resolvers, coverage and loop-aware reasoning **16**
B.1 A resolver is a checked evidence-producing computation . . . 16
B.2 Reading a table gives candidates; coverage gives completeness . . . 16
B.3 The implemented loop rule . . . 16

C Questions and evidence-indexed claims **17**

D Exact compact completion changes representation, not the problem **18**

E Separate supplied-stateflow and LLVM IR families **18**
E.1 Finite abstract stateflow . . . 18
E.2 Direct verified LLVM IR . . . 19

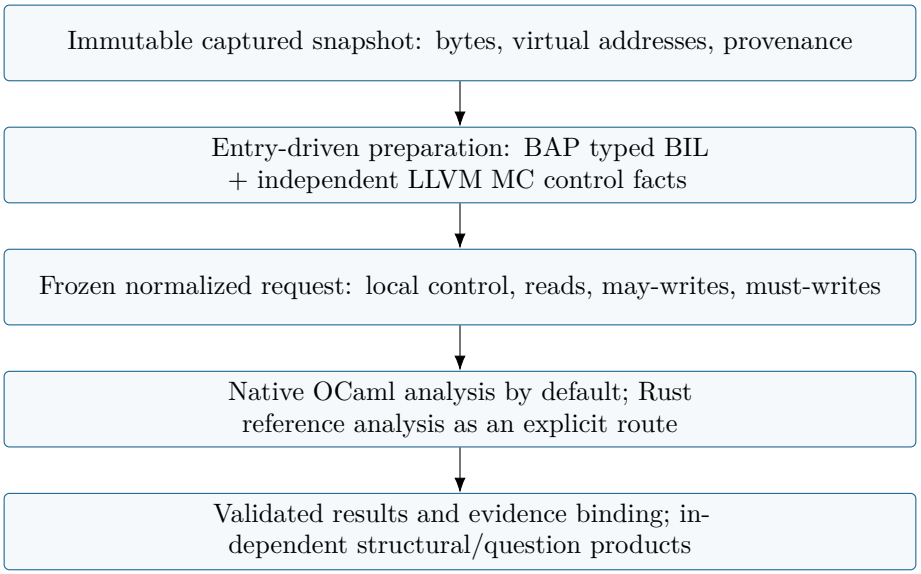
F Current evidence and remaining scope **19**

G Source map, origin and direct follow-up **20**

1 Purpose and module boundary

Ariadne's core answers: *given these roots, these admitted instruction summaries and these bytes, which local edges and possible data origins follow?* The machine being modeled is the *analyzer*; its state is not a recovered historical CPU state.

The production minidump route has the following ownership chain:



Preparation owns discovery and semantic admission. The core owns recovery, reaching definitions and slicing over the frozen request. Investigation owns bound, question-specific claims. Protocol validation connects the native helper to the Rust host. These responsibilities have different proof boundaries.

The structural frame route branches before full completion: it seals a recovery prefix, including a drained one, while the core phase is still `Recover`. It can feed frame connections and focused questions without computing the full reaching-definition table.

Scope. Windows/Linux AMD64 minidumps are implemented; general PE/ELF image readers and ELF-core readers are deferred. BAP lifting is a pinned trusted dependency. The independent AMD64 instruction-step model is retired. Supplied LLVM IR and supplied abstract stateflow are separate analysis paths, not implicit minidump execution semantics.

2 Concrete implementation

Carrier or module	What the implementation actually holds or does
`AnalysisRequest`	Owned snapshot ID, finite address/location sets, entry roots, slice seeds, byte-source sets, successful-decode set and an instruction map total on candidate addresses.
`Instruction`	Kind, continuation set, known target set, completeness Boolean, uses, must-defs and may-defs. It is an admitted summary, not a numeric CPU state.
`AnalysisState`	Phase, pending/visited/decoded sets, provenance map, unified typed edge set, obligations, per-site reaching sets and slice.

Carrier or module	What the implementation actually holds or does
`Analyzer`	Owns the validated request and mutable state. Indices, caches and work counters accelerate analysis. Ordinary `step` selects the lowest eligible VA.
`DefinitionSet`	Either an ordered tree or shared sorted definition universe with immutable membership bits. Mutating a dense set creates a private tree representation.
Native core	OCaml helper owns production analysis state. Rust checks request/source identity, protocol, pages, result domains and clean completion.
`CompletedAnalysis`	Private request/state carrier; immutable state is shared through `Arc`. It implements sealed `AnalysisView`.
`BoundFrameStructure`	Private owner of a captured frame chain, per-activation bound recovery prefixes, links, maps and diagnostics.
`BoundFrameConnections`	Owns its structural source and a read-only derived graph assessment.
`BoundFrameQuestion`	Binds one typed question, admitted evidence, source revision, independent layer assessments and work receipts.

Mutation and ownership. The request is validated at construction and then exposed through shared references. Analysis mutates worklists and facts, not the request. Later preparation revisions are separate frozen problems, not a mutation of an already analyzed request. `Arc` shares immutable results; it does not turn one captured snapshot into evidence of concurrent program execution.

Effects and errors. The pure mathematical core consumes supplied facts. The surrounding pipeline reads capture files and exchanges bounded messages with helper processes. Invalid requests and malformed proofs are errors; unknown targets, unsupported semantics and exhausted question work remain explicit gaps or layer stops under their respective product contracts.

3 Concrete invariants and their enforcement

A record can have the correct field types and still violate the analysis contract. The important obligations are stronger than record shape.

Invariant	Principal enforcement
Snapshot ID and roots are nonempty; instruction map has exactly the candidate domain.	`AnalysisRequest::validate`; TLA+ assumptions and address contract.
Targets and continuations stay in that domain; uses/may-defs stay in the location domain.	Request validation.
Must-defs are a subset of may-defs; continuation/target cardinalities match instruction kind.	Request validation.
Only discovered local successors are queued; visited and pending are disjoint.	Construction and transition code; recovery invariants.

Invariant	Principal enforcement
Call edges remain visible but do not transmit local dataflow into callees.	Explicit edge-kind allowlist, transition equations and conformance controls.
Definition origins name a declared entry or a decoded possible writer.	Engine construction, native result checks and model invariants.
Slicing begins at the fixed point; done results are dependency-closed.	Phase guards and completion/result validation.
Native results bind the exact request, helper and transaction.	Host protocol/session validation; not Rust's field types alone.
Captured observations, derived facts and hypotheses retain distinct authority.	Evidence constructors, source/proof validation and report contracts.
A true-looking target list also covers every allowed incoming case.	Resolver coverage validation; a list or hash alone is insufficient.

Some boundaries remain premises: correct BAP lifting, conservative alias normalization, the relationship between captured memory and an earlier instruction, and the distinct identity assigned to each snapshot. Finite model checks and tests do not prove these premises universally.

4 Representative execution in the code

Consider one tracked location x and four decoded sites in a single snapshot:

$$a_0 \xrightarrow{\text{taken}} a_1 \longrightarrow a_3, \qquad a_0 \xrightarrow{\text{fallthrough}} a_2 \longrightarrow a_3.$$

The only root is a_0 and the only slice seed is a_3. At a_1, a definite write has $\mathsf{must} = \mathsf{may} = \{x\}$. At a_2, a possible write has $\mathsf{must} = \varnothing$, $\mathsf{may} = \{x\}$. At a_3, the instruction reads x and ends the local scope. All unmentioned tracked uses and writes are empty.

`Analyzer::new` validates the request and queues a_0. `step`/`visit` discovers both branch edges and then their continuations. Neither the slice seed nor a crash-time value selects a branch. After `FinishRecovery`, propagation computes:

$$\begin{aligned} R(a_0) &= \{\mathrm{Entry}(a_0, x)\}, \\ \mathrm{Out}(a_1) &= \{\mathsf{Write}(a_1, x)\}, \\ \mathrm{Out}(a_2) &= \{\mathrm{Entry}(a_0, x), \mathsf{Write}(a_2, x)\}, \\ R(a_3) &= \{\mathrm{Entry}(a_0, x), \mathsf{Write}(a_1, x), \mathsf{Write}(a_2, x)\}. \end{aligned}$$

`FinishDataflow` starts the slice at $\{a_3\}$. `ExpandSlice` adds a_1, a_2. Entry origins remain reportable unknown inputs but are not producer instruction nodes. The final slice is $\{a_1, a_2, a_3\}$; it excludes a_0 because this is a data slice, not a control-dependency slice. The result says which origins are possible under the supplied summaries, not which predecessor executed.

5 First abstraction boundary

From concrete storage to the analysis machine we preserve snapshot identity, semantic VAs, finite request domains, instruction effects, typed edges, provenance, obligations, phase boundaries, definitions and slice membership. We erase allocator behavior, `BTreeMap` layout, internal indices,

cache contents and membership-word placement. Resource policy is retained as a separate observable outcome; exact wall-clock behavior is not part of the unbounded core transition system.

This abstraction deliberately does not invent numeric register values from uses/may-defs/must-defs. Those sets describe dependencies and writes. A numeric transfer needs separately admitted BIL semantics or a supplied stateflow relation.

6 The minimal abstract machine

Fix a valid immutable request q. Write A for candidate VAs, L for locations, E_0 for entry roots, Z for slice seeds and I for instruction summaries. Let Δ_q be the finite universe of possible definition identities. An analyzer configuration is

$$\Sigma = \langle p, P, V, D, \pi, G, O, R, S \rangle_q.$$

Component	Meaning
p	Phase: recover, dataflow, slice or done
$P, V, D \subseteq A$	Pending, visited, successfully decoded starts
$\pi : A \to \mathsf{ByteSource}$	Whole-instruction selected-source provenance
$G \subseteq \mathsf{Edge}(A)$	All typed edges, including call edges
$O \subseteq \mathsf{Obligation}(A)$	Explicit unresolved recovery/target obligations
$R : A \to \mathcal{P}_f(\Delta_q)$	Possible origins immediately before each instruction
$S \subseteq D$	Backward instruction data slice

The immutable subscript q is essential: a changed graph or premise set requires a new bound analysis problem. R is a map of origins, not a map of machine integers. G may contain edges whose destination is not decoded. An unavailable destination remains a recovery boundary rather than becoming a fabricated terminal instruction.

Initialization sets $P = E_0$, $p = \mathsf{recover}$, every $R(a) = \varnothing$ and all other set components empty; initially $\pi(a) = \mathsf{unavailable}$.

7 Abstract operations

The main transformations are:

$$\begin{aligned}
\mathsf{validate} &: \mathsf{RawRequest} \to \mathsf{ValidRequest} + \mathsf{InvalidRequest},\\
\mathsf{visit} &: \mathsf{RecoverState} \times A \rightharpoonup \mathsf{RecoverState},\\
\mathsf{propagate} &: \mathsf{DataflowState} \times D \rightharpoonup \mathsf{DataflowState},\\
\mathsf{expand} &: \mathsf{SliceState} \rightharpoonup \mathsf{SliceState},\\
\mathsf{finishBounded} &: \mathsf{Analyzer} \times \mathsf{Limits} \to \mathsf{CompletedResult} + \mathsf{CompletionError}.
\end{aligned}$$

A partial operation is enabled only under the relevant phase and work guard. Native completion additionally crosses transport/validation operations; a successful helper response is not yet an admitted public result.

Preparation has a different operation:

$$\mathsf{prepare} : \mathsf{Snapshot} \times \mathsf{Query} \times \mathsf{Policy} \to \mathsf{FrozenRequestWithEvidence} + \mathsf{PreparationError}.$$

It may discover candidate starts and propose guarded/target refinements before freezing the final request. That earlier discovery does not replace the core's subsequent recovery pass.

8 Reachability and invariants

Representable records strictly exceed states reachable from initialization. For example, a public `AnalysisState` record can be assembled with phase `Done` and inconsistent reaching sets. Field types allow it; a completed analysis constructor must not admit it.

For valid reachable core states:

$$\begin{aligned} P \cap V &= \varnothing, & D \subseteq V \subseteq A, \\ P \cup V &= E_0 \cup \{e.\mathsf{dst} \mid e \in G, \mathrm{local}(e)\}, \\ G &= \bigcup_{a \in D} \mathsf{InstructionEdges}(I(a)), \\ p \neq \mathsf{recover} &\Longrightarrow P = \varnothing. \end{aligned}$$

Moreover $R(a) = \varnothing$ for undecoded a; decoded rows contain only valid entry/writer identities. In slice/done phases all reaching equations are at the fixed point. At done the slice contains decoded seeds and is closed under its data producers.

These are reconstructed semantic refinements. TLA+ names the principal checks `TypeOK`, `RecoveryInvariant`, `DefinitionInvariant` and `ResultInvariant`. Their finite checking and Rust/native conformance are separate evidence, not a proof that every concrete state transition refines this presentation.

9 Abstract execution trace

For the diamond of Section 4, the permitted sequence has the shape

$$\begin{aligned} &\Sigma_0 \xrightarrow{\mathsf{Visit}(a_0)} \Sigma_1 \xrightarrow{\mathsf{Visit}(a_1)} \Sigma_2 \xrightarrow{\mathsf{Visit}(a_2)} \Sigma_3 \xrightarrow{\mathsf{Visit}(a_3)} \Sigma_4, \\ &\Sigma_4 \xrightarrow{\mathsf{FinishRecovery}} \Sigma_5 \xrightarrow{\mathsf{Propagate}^*} \Sigma_6 \xrightarrow{\mathsf{FinishDataflow}} \Sigma_7, \\ &\Sigma_7 \xrightarrow{\mathsf{ExpandSlice}} \Sigma_8 \xrightarrow{\mathsf{FinishSlice}} \Sigma_9. \end{aligned}$$

The star denotes finitely many enabled propagation actions. The specification allows different eligible-site orders. Ordinary Rust replay chooses its fixed lowest-VA order; completion-only reverse postorder changes work order while preserving the intended final fixed point. Equal final states do not imply equal intermediate replay traces.

If recovery also reaches an unavailable a_4, its incoming edge and $\mathsf{Unavailable}(a_4)$ obligation survive into Σ_9. Done then means the algorithm completed its admitted finite problem; it does not erase that gap.

10 Second abstraction boundary

The formal presentation below replaces the machine's raw record shape with structural records, variants and semantic refinements. It preserves all components needed for the equations and judgments. It erases concrete Rust and OCaml constructors, message framing, cache invalidation mechanics, numeric counter encodings and serialization syntax.

We index types by a snapshot, request or activation where those identities matter. This is an explanatory strengthening of the type notation. The actual code often stores these indices as strings/integers and checks compatibility at runtime. There is no claim that Rust statically proves the indices equal.

11 Abstract syntax and structural types

11.1 The notation and its limits

We use an extensional structural calculus:

$$\tau ::= \mathsf{Bool} \mid \mathsf{Word}_{64} \mid \mathsf{String} \mid \{\, \ell_i : \tau_i \,\}_{i \in J} \mid \langle k_i : \tau_i \rangle_{i \in J} \mid \mathcal{P}_f(\tau) \mid (\tau \to \sigma) \mid \{x : \tau \mid \phi(x)\}.$$

Records give named fields; variants give tagged alternatives. $\mathcal{P}_f(\tau)$ is the type of finite sets. $\{x : \tau \mid \phi\}$ is a semantic refinement, not a statement that an arbitrary host-language record proves ϕ.

For immutable observations, record width subtyping forgets extra fields:

$$\frac{J \subseteq K \qquad \forall j \in J.\ \tau_j <: \sigma_j}{\{\, \ell_k : \tau_k \,\}_{k \in K} <: \{\, \ell_j : \sigma_j \,\}_{j \in J}}\ \textsc{Record}.$$

This permits using a richer evidence record where a smaller read-only view is required. It does not permit strengthening an unknown fact, dropping its premises, or assigning mutable Rust records new subtype relationships. A view that forgets premise fields does not gain the authority to publish an unconditional claim; publication still crosses the owning validator. General subtyping notation follows Pfenning's lecture notes; the Ariadne instantiations here are our reconstruction.[1]

11.2 Identity and instruction structure

Let $s : \mathsf{SnapshotId}$ be nonempty and $A_s \subseteq \mathsf{Word}_{64}$.

$$\begin{aligned}
\mathsf{Addr}(s) &\equiv \{\, \mathsf{snapshot} : \{s\}, \mathsf{va} : A_s \,\},\\
\mathsf{Loc}(q) &\equiv L_q,\\
\mathsf{Def}(q) &\equiv \{\, \mathsf{loc} : L_q, \mathsf{site} : A_q, \mathsf{origin} : \mathsf{Entry} + \mathsf{Instruction} \,\},\\
\mathsf{Edge}(q) &\equiv \{\, \mathsf{src} : A_q, \mathsf{dst} : A_q, \mathsf{kind} : \mathsf{EdgeKind} \,\}.
\end{aligned}$$

The actual core stores bare VAs inside a request carrying one snapshot ID. The indexed address notation makes that implicit common context visible. A file offset and a graph node ID are different semantic types even if all three use integers in an implementation.

$$\begin{aligned}
\mathsf{Insn}(A, L) \equiv \{\, &\mathsf{kind} : \mathsf{Ordinary} + \mathsf{Conditional} + \mathsf{Jump} + \mathsf{Indirect}\\
&\quad + \mathsf{Call} + \mathsf{Return} + \mathsf{Stop},\\
&\mathsf{fall} : \mathcal{P}_f(A),\ \mathsf{targets} : \mathcal{P}_f(A),\ \mathsf{complete} : \mathsf{Bool},\\
&\mathsf{uses} : \mathcal{P}_f(L),\ \mathsf{must} : \mathcal{P}_f(L),\ \mathsf{may} : \mathcal{P}_f(L) \,\}.
\end{aligned}$$

This is close to the concrete record. Its cross-field constraints are given by well-formedness, rather than encoded as distinct Rust constructors.

11.3 A request is a refined record

$$\begin{aligned}
\mathsf{RawRequest} \equiv \{\, &\mathsf{snapshot} : \mathsf{String},\ A : \mathcal{P}_f(\mathsf{Word}_{64}),\ L : \mathcal{P}_f(\mathsf{String}),\\
&E_0 : \mathcal{P}_f(A),\ Z : \mathcal{P}_f(A),\ \mathsf{input} : \mathsf{Binary} + \mathsf{Dump},\\
&C : \mathcal{P}_f(A),\ F : \mathcal{P}_f(A),\ T : \mathcal{P}_f(A),\ D_0 : \mathcal{P}_f(A),\\
&I : A \to \mathsf{Insn}(A, L) \,\}.
\end{aligned}$$

[1] Frank Pfenning, *Lecture Notes on Subtyping*, 15-312, 2003: https://www.cs.cmu.edu/~fp/courses/15312-f03/handouts/16-subtyping.pdf.

Here C, F, T, D_0 mean captured, file-backed, trusted fallback and decodable. The display uses dependent fields as shorthand; the raw Rust representation uses unrestricted sets/maps before validation.

$$\mathsf{ValidRequest} \equiv \{q : \mathsf{RawRequest} \mid \mathrm{WF}(q)\}.$$

The semantic domain is refined. The real `AnalysisRequest` type itself still admits invalid defaults; `Analyzer::new` checks them.

12 Bindings and scopes

Ariadne's core does not resolve source-language lexical names. A location string is an already normalized storage/dependency identifier. Architectural aliases such as EAX/RAX or vector subviews must have been projected into compatible tracked locations by the effects layer.

Entry definitions are *scope-qualified unknown inputs*:

$$\mathrm{Entry}(a, \ell) = (\ell, a, \mathsf{Entry}), \qquad \mathsf{Write}(a, \ell) = (\ell, a, \mathsf{Instruction}).$$

The tag prevents a write at an entry VA from being mistaken for the entry input at that VA. Different entry roots receive different entry identities.

Frames add a second scope:

$$\mathsf{Node}(s, \alpha) = \{\, \mathsf{activation} : \alpha, \mathsf{va} : \mathsf{Word}_{64} \,\}.$$

The implemented node is `(frame, site)` inside a bound captured chain; s, α are its implicit owning context. Two recursive activations with the same instruction VA are different nodes. One static node may still have many loop visits. A bounded model-prefix occurrence identifies a modeled visit, not a recovered historical iteration number.

13 Judgment forms

Every judgment used below has a specific scope:

Judgment	Meaning
$\Gamma \vdash v : \tau$	The mathematical value has structural/refinement type τ under declared context Γ.
$\Gamma \vdash q\ \mathsf{ok}$	The normalized request satisfies its input contract.
$q \vdash \Sigma\ \mathsf{valid}$	Machine shape and phase-sensitive invariants hold.
$q \vdash \Sigma \xrightarrow{\alpha} \Sigma'$	An enabled analyzer action transforms this fixed problem.
$\Gamma; \Pi \vdash c\ \mathsf{admitted}$	Claim c has passed its owning validator under premises Π and exact bound source.
$\Gamma; \Pi \vdash \rho\ \mathsf{covers}(a)$	A checked resolver receipt covers the allowed incoming cases at dispatch a for its bound graph epoch.

Γ is a mathematical context of identities, domains and admitted contracts. It is not a new runtime module named Gamma. Π records semantic premises/hypotheses. The last two judgments summarize several real validation operations; they are not existing proof-assistant theorem names.

14 Statics: validity and permission to make a claim

14.1 Request well-formedness

The following schema summarizes `AnalysisRequest::validate`. First define the per-instruction refinement:

$$\mathsf{InsnOK}_{A,L}(i) \equiv i.\mathsf{must} \subseteq i.\mathsf{may} \subseteq L \ \wedge \ i.\mathsf{uses} \subseteq L \\ \wedge \ i.\mathsf{fall} \subseteq A \ \wedge \ i.\mathsf{targets} \subseteq A \ \wedge \ \mathsf{Shape}(i).$$

Then the request rule is

$$\frac{s \neq \epsilon \quad A \neq \varnothing \quad E_0 \neq \varnothing \\ E_0, Z, C, F, D_0 \subseteq A \quad T \subseteq F \quad \mathrm{dom}(I) = A \\ \forall a \in A.\ \mathsf{InsnOK}_{A,L}(I(a))}{\Gamma \vdash q\ \mathsf{ok}}\ \textsc{Request}.$$

Shape requires exactly one continuation for ordinary, conditional and call instructions; none for other kinds. Ordinary/return/stop have no targets; conditional/direct jump have exactly one target; indirect/call allow finite supplied target sets. Empty L and Z are legal.

These rules check the consistency of a supplied semantic summary. They do not prove it matches machine bytes or hardware behavior.

14.2 Admission has more premises than shape

A schematic target-completion rule is

$$\frac{\mathsf{SourceBound}(\rho, s, e) \quad \mathsf{ReplayValid}(\rho) \\ \Gamma; \Pi \vdash \rho\ \mathsf{covers}(a) \quad \mathsf{Targets}(\rho) = I(a).\mathsf{targets}}{\Gamma; \Pi \vdash I(a).\mathsf{complete} = \mathsf{true}\ \mathsf{admitted}}\ \textsc{Target-Coverage}.$$

e is the candidate graph epoch. This rule is deliberately stronger than "a supported table expression was recognized." Recognizing a pattern permits an attempt; admitting completeness requires covered entrances, transfers, table data and successors. The core itself trusts the supplied `complete` flag; the production preparation/admission boundary owns the meaning of that flag.

Likewise a well-shaped hypothesis is not an observation. Conditional normal selection can be described as

$$\Gamma; \Pi \cup \{h : \mathsf{AlignedAt}(s, \alpha, a, 16)\} \vdash \mathsf{NormalContinuation}(a)\ \mathsf{conditional}.$$

It must retain h and the baseline unknown alignment verdict. It supplies no observed RSP and no earlier executed branch.

15 Dynamics: recovery, fixed point and slice

15.1 Selected bytes and recovered control

For dump input, whole-instruction source selection is

$$\mathsf{Source}_q(a) = \begin{cases} \mathsf{Captured}, & a \in C, \\ \mathsf{File}, & a \notin C \wedge a \in T, \\ \mathsf{Unavailable}, & \text{otherwise.} \end{cases}$$

For binary input use file bytes only when $a \in F$. Production minidump input is capture-only; the generic core's fallback contract does not authorize that reader to fill a hole. A captured decode failure never retries with file bytes. Let $\mathsf{CanDecode}(a)$ mean a selected source is available and $a \in D_0$.

For a successful visit, let G_a be all instruction edges, O_a its target obligations and $\mathsf{succ}_{loc}(a)$ its local successors. Then

$$\frac{p = \mathsf{recover} \quad a \in P \quad \mathsf{CanDecode}(a)}{\langle p, P, V, D, \pi, G, O, R, S\rangle_q \xrightarrow{\mathsf{Visit}(a)} \Sigma'} \textsc{Visit-OK},$$

where

$$V' = V \cup \{a\}, \qquad D' = D \cup \{a\}, \qquad P' = (P \cup \mathsf{succ}_{loc}(a)) \setminus V',$$
$$G' = G \cup G_a, \qquad O' = O \cup O_a, \qquad \pi' = \pi[a \mapsto \mathsf{Source}_q(a)].$$

Unmentioned fields do not change. A failed visit removes a from P, marks it visited and adds unavailable/decode-failed; it adds no decoded node or fabricated effects. Incoming edges to that failed endpoint remain.

Summary kind	Edges added by successful recovery
Ordinary	$a \xrightarrow{next} b$
Conditional	$a \xrightarrow{taken} t$ and $a \xrightarrow{fallthrough} b$
Jump	$a \xrightarrow{jump} t$
Indirect	$a \xrightarrow{indirect} t$ for every supplied t
Call	$a \xrightarrow{call} f$ for each callee; $a \xrightarrow{summary} b$
Return / stop	No local continuation

The local edge allowlist is next, taken, fallthrough, jump, indirect, summary. Call edges are excluded from local discovery and from the dataflow predecessor relation. Unknown indirect/call targets produce obligations whenever their summary has $\mathsf{complete} = \mathsf{false}$.

15.2 Reaching definitions are a monotone finite problem

For decoded a, write $R(a)$ for the input origins before a:

$$\mathrm{Gen}(a) = \{\mathsf{Write}(a, \ell) \mid \ell \in I(a).\mathsf{may}\},$$
$$\mathrm{Out}_R(a) = \{d \in R(a) \mid d.\mathsf{loc} \notin I(a).\mathsf{must}\} \cup \mathrm{Gen}(a),$$
$$\mathrm{Preds}(a) = \{p \in D \mid \exists e \in G.\ \mathrm{local}(e) \wedge e.\mathsf{src} = p \wedge e.\mathsf{dst} = a\},$$
$$\mathrm{Incoming}_R(a) = \begin{cases} \{\mathrm{Entry}(a, \ell) \mid \ell \in L\}, & a \in E_0, \\ \varnothing, & a \notin E_0 \end{cases} \cup \bigcup_{p \in \mathrm{Preds}(a)} \mathrm{Out}_R(p).$$

$$\frac{p = \mathsf{dataflow} \quad a \in D \quad \mathrm{Incoming}_R(a) \not\subseteq R(a)}{\Sigma \xrightarrow{\mathsf{Propagate}(a)} \Sigma[R(a) \mapsto R(a) \cup \mathrm{Incoming}_R(a)]} \textsc{Propagate}.$$

Define $F_q(R)(a) = \mathrm{Incoming}_R(a)$. The solution reached from bottom is

$$R^* = \mathrm{lfp}(F_q) \quad \text{in} \prod_{a \in D} \mathcal{P}_f(\Delta_q).$$

Must-defs can kill old origins in a transfer, but the kill set is fixed, so $X \subseteq Y$ implies $\mathrm{Out}_X(a) \subseteq \mathrm{Out}_Y(a)$. May-defs alone add alternatives and preserve old ones. Cycles are handled by monotone growth in a finite lattice, not by enumerating loop executions.

15.3 Phase boundaries and the backward closure

$$\frac{p = \mathsf{recover} \quad P = \varnothing}{\Sigma \xrightarrow{\mathsf{FinishRecovery}} \Sigma[p \mapsto \mathsf{dataflow}]}, \qquad \frac{p = \mathsf{dataflow} \quad R = F_q(R)}{\Sigma \xrightarrow{\mathsf{FinishDataflow}} \Sigma[p \mapsto \mathsf{slice}, S \mapsto Z \cap D]}.$$

For sliced instruction a,

$$\begin{aligned} \mathrm{Deps}_R(a) &= \{d \in R(a) \mid d.\mathsf{loc} \in I(a).\mathsf{uses} \wedge d.\mathsf{origin} = \mathsf{Instruction}\}, \\ H_R(S) &= S \cup \{d.\mathsf{site} \mid a \in S,\ d \in \mathrm{Deps}_R(a)\}. \end{aligned}$$

Expand while $H_R(S) \neq S$. If $H_R(S) = S$, change phase from slice to done. The resulting slice includes dependencies of *all uses* of every included instruction; it need not be a minimal dependency set for one output operand. The branch instruction in our diamond is therefore not automatically included.

16 Errors, unknowns and partial results

The following sum makes the conceptual differences explicit:

$$\mathsf{Outcome} = \mathsf{Rejected}(\mathsf{Error}) + \mathsf{Stopped}(\mathsf{LayerReceipt}) + \mathsf{Admitted}(\mathsf{ScopedProduct}).$$

This is a semantic organization, not one existing public Rust enum. Each API has its own result type and admission rules.

- **Invalid input or forged evidence**: refuse admission. Exhaustion must not hide malformed identity, schema or proof data.
- **Completed core with obligations**: valid completed partial analysis. The equations are solved on the recovered graph and the gaps survive.
- **Bounded core completion failure**: `finish_bounded` returns `CompletionError`; it does not publish a stopped prefix as completed.
- **Structural prefix**: an independently valid product even without dataflow. It is not a disguised completed core result.
- **Question or connection work stop**: retain admitted facts and layer-specific unknowns/receipts. A stopped search cannot justify absence.
- **Verifier exhaustion**: refuse a trusted result rather than accepting unverified producer claims.

The implemented core predicate is exactly

$$\mathsf{scopeClosed}(r) \equiv (r.\mathsf{phase} = \mathsf{Done}) \wedge (r.\mathsf{obligations} = \varnothing).$$

It is relative to the normalized request and traversal policy. Missing seeds are reported separately as $Z \setminus D$. Scope closure implies neither all real paths nor complete callee execution nor recovery of execution history.

17 Implementation correspondence

Formal construct	Concrete owner or source symbol
Insn, ValidRequest	`src/model.rs`: `Instruction`, `AnalysisRequest::validate`.
Σ, Visit, phase steps	`Specs/Ariadne.tla`: `Init`, `Visit`, `Next`; `src/engine.rs`: `Analyzer::step`, `visit`.
F_q, H_R	`Specs/Ariadne.tla`: `Out`, `Incoming`, `Dependencies`; Rust engine and native `recovery.ml`.
Compact fixed-point storage	`src/completion.rs`, `src/definition_set.rs`, native `dense_completion.ml`.
Transport admission	`src/bap/core_protocol.rs`, `core_session.rs`, `core_adapter.rs`.
Snapshot reads	`src/input/address_space.rs`: `ByteRead`, `ReadStop`; `materialize.rs`: preparation.
covers judgment	`src/bap/target_resolution.rs`; private `target_resolution/loop_closure.rs`; `guard_admission.rs`; combined materialization.
Conditional alignment	`src/bap/alignment_hypothesis.rs`; `Specs/AriadneAlignmentHypothesis.tla`.
Bound recovery and frame structure	`src/recovery.rs`, `src/investigation/frame_structure.rs`; `Specs/AriadneFrameStructure.tla`.
Corridor and SCC	`src/investigation/frame_connections.rs`; `Specs/AriadneFrameConnections.tla`.
Typed question and values	`src/investigation/frame_question/`; `memory_refinement.rs`; `backward_values.rs`.
Completed read-only view	`src/analysis_view.rs`: sealed `AnalysisView`, private `CompletedAnalysis`.
Separate stateflow / IR	`src/machine_state.rs`, `src/llvm_ir.rs`; their separate TLA+ models.

The core transition equations closely correspond to the specification. The admission, coverage and indexed claim rules are higher-level summaries of multiple validators. The paper's dependent indices and width-subtyping rule are explanatory constructs, not host-language declarations.

18 Final abstraction boundary

This reconstruction ignores concrete instruction encodings, full numeric CPU semantics, OS scheduling, exception-handler execution, allocator layout, network/filesystem scheduling and timing. The core treats byte selection, decodability and instruction effects as fixed inputs. Resolver/question layers recover selected semantics only under their own supported contracts.

Assumptions made visible rather than proved here include: snapshot identity is meaningful; the admitted lifts have their declared semantics; locations conservatively account for aliases; a captured-control-memory premise is appropriate for the selected question; and explicit hypotheses are acceptable conditions for a conditional result. A content hash binds bytes internally; it does not authenticate capture provenance or prove historical stability.

19 Model versus implementation: what the types do not prove

19.1 Excess representable states

TLA+ addresses range over nonnegative mathematical integers; the Rust implementation deliberately restricts them to 64-bit VAs. All sets used by the analysis request are finite. These representation choices are part of the correspondence obligation.

Rust aliases such as `Address = u64` and `Location = String` do not prevent an accidental file-offset/VA mixup by themselves. A `Default::default()` request has invalid empty fields. Public instruction records can have contradictory kind/target combinations. A completion flag can be written without a coverage receipt in raw structures. These facts explain why construction boundaries and validators matter.

Private completed/bound carriers and a sealed read-only trait restrict ways external callers can fabricate evidence-bearing objects. They do not turn all semantic contracts into compiler-checked theorems. In particular, `AnalysisView` also admits an in-progress `Analyzer`; consumers must still enforce the phase they require.

19.2 Preservation and progress, with their boundaries

The intended preservation statement is

$$q\ \mathsf{ok} \wedge q \vdash \Sigma\ \mathsf{valid} \wedge q \vdash \Sigma \longrightarrow \Sigma' \implies q \vdash \Sigma'\ \mathsf{valid}.$$

For finite valid core requests under the model's fair scheduling, recovery visits finitely many starts, dataflow adds finitely many facts and slicing adds finitely many nodes. Thus the unbounded abstract analyzer terminates. This argument does not promise a small runtime or prevent concrete resource exhaustion. It is distinct from termination of the analyzed program.

TLA+ finite scenarios, generated replay, independent oracles and compiled mutation controls test these obligations on declared corpora. They do not establish universal Rust/OCaml refinement or hardware-level ISA soundness. This document adds no machine-checked metatheorem.

19.3 Which questions are computable here?

A fixed finite graph's reachability, SCCs and reaching-definition least fixed point are decidable. Simple-path enumeration is finite but can be enormous; unbounded loop walks may be infinite. General program reachability can be undecidable under an unbounded-program model. An exact historical path can also be underdetermined because one snapshot is compatible with several histories. These are different reasons for not recovering one unique path.

Current Ariadne addresses them with bounded work, explicit supported semantic profiles, induction for selected loops, independent layer statuses and question-relative claims. A positive graph path is structural evidence. A checked model witness is model-relative evidence. Neither is a recording of the path actually executed before the dump.

A Frame products as a family of structural interfaces

The overall system is best modeled as a family of products, not one Boolean called complete:

$$\begin{aligned}\mathsf{Product} = {} & \mathsf{CoreDone} + \mathsf{FrameStructure} + \mathsf{FrameConnections} \\ & + \mathsf{FrameQuestion} + \mathsf{FramePaths} \\ & + \mathsf{SuppliedStateflow} + \mathsf{SuppliedIR},\end{aligned}$$

$$\begin{aligned}\mathsf{ReportStatus} \equiv \big\{ & \mathsf{binding} : B,\ \mathsf{recovery} : R_c,\ \mathsf{connection} : C_c, \\ & \mathsf{question} : Q_c,\ \mathsf{origins} : O_c,\ \mathsf{history} : H_c \big\}.\end{aligned}$$

The second record is a semantic summary of independent dimensions, not a single serialized schema shared by every product.

A.1 Stage 5.1: publish structure before full dataflow

A recovery prefix conceptually has type

$$\mathsf{RecoveryPrefix}(q) = \{\Sigma : \mathsf{State}(q) \mid p = \mathsf{Recover} \wedge \mathsf{PrefixValid}(q, \Sigma)\}.$$

Its stop is drained, work-limited or deadline-limited under the owning recovery contract. Even a drained prefix stays in Recover: no `FinishRecovery`, dataflow or slicing is credited. A failed or pending endpoint remains a gap attached to the original edge.

Frame structure binds one such prefix per independently recovered activation. A captured return address helps identify the active call/continuation under admitted unwind and byte evidence. It does not identify all prior branch outcomes or all prior values of volatile registers.

A.2 Stage 5.2: corridor, SCCs and bounded examples

The represented route graph contains activation-qualified decoded nodes and admitted local edges whose endpoints lie in the selected function extents. Call-only destinations are context. A cross-frame transition uses a validated active-call link. At an intermediate active-call anchor, the matched edge enters the child while the summary continuation remains context.

For admitted start u and endpoint v, completed graph traversals yield

$$F = \mathsf{Reach}_G(u), \quad B = \mathsf{Reach}_{G^{-1}}(v), \quad K = F \cap B.$$

K is the represented corridor. Strongly connected components summarize cycles without enumerating their walks. The condensation graph is acyclic. The iterative SCC implementation and bounded BFS/detour examples operate on this graph, not on a recovered CPU execution.

Its actual connection verdict is a variant:

$$\begin{aligned}\mathsf{RepresentedConnection} = {} & \mathsf{Found} + \mathsf{AbsentInRepresentedGraph} \\ & + \mathsf{UnknownWorkStopped} + \mathsf{UnavailableAnchors}.\end{aligned}$$

Absence requires completed graph construction and reachability. It remains absence only in that represented graph. A forward-reachable unresolved jump outside K may reconnect later, so its frontier must be retained too. SCC completion, corridor completion and example-search completion are separate statuses. Requested example count NP is bounded; retained examples do not establish a total path count.

A.3 Full frame paths are a separate product

The frame-path route composes admitted frames and uses typed-edge bounded DFS with visit/work/step constraints. It can need completed per-frame analyses and additional compatibility evidence. Its enumeration-exhausted field must be interpreted together with visit limits, graph gaps and semantic scope. The existence of a Stage 5.2 route does not by itself close the full frame-path acceptance gate, prove feasibility or establish execution history.

B Resolvers, coverage and loop-aware reasoning

B.1 A resolver is a checked evidence-producing computation

A useful semantic interface is

$$\mathsf{resolve} : \mathsf{BoundSource} \times \mathsf{DispatchQuestion} \times \mathsf{Limits} \to \mathsf{CandidateEvidence} + \mathsf{Unknown} + \mathsf{Error}.$$

This is an explanatory interface; current Ariadne has private cooperating modules and one preparation coordinator, not a generic resolver registry. The code's pattern recognition is over typed transfer/control structure and captured-read witnesses, not a claim that an opcode name proves a pattern. Guard admission, ordinary target evaluation and loop-coverage checking have related but different responsibilities.

A resolution record contains site, finite targets, a complete flag, ordinary path receipts, unresolved reasons and optional coverage evidence. The implemented coverage variant is

$$\mathsf{CoverageEvidence} = \mathsf{ExistingPrefixTree}(\mathsf{ReceiptRefs}) + \mathsf{InductiveDispatch}(\mathsf{LoopClosureReceipt}).$$

Thus loop awareness extends the existing resolver's coverage strategy. It is not selected as a competing plugin when another pattern also matches.

B.2 Reading a table gives candidates; coverage gives completeness

For a captured relative-32 table, the recognized target expression is schematically

$$t = \left(b + \mathrm{sext}_{32\to 64}(M_s[b + 4i])\right) \bmod 2^{64}.$$

Here b is a proved base, i is a bounded selector and M_s denotes source-bound captured control-data bytes under the explicit `FrozenCapturedControlData` premise. Hole/conflict/overflow and unsupported expression forms remain explicit. Captured bytes do not automatically prove what a mutable heap location held at an earlier execution point.

Finding one witness gives one admitted candidate. Certifying the full target set requires all permitted selectors/incoming cases, relevant control coverage and exact read evidence. A partial list cannot be made complete by truncating a domain, assuming an unproved predecessor impossible, or setting a Boolean on the producer side.

B.3 The implemented loop rule

For a finite local dispatch region H, let J_n be a point assertion and $[\![e]\!]$ the supported width-aware transfer. Coverage checks have the form

$$\begin{aligned} \mathsf{EntranceState}(e) &\subseteq J_{e.\mathsf{dst}}, && \text{every region entrance,} \\ [\![e]\!](J_{e.\mathsf{src}}) &\subseteq J_{e.\mathsf{dst}}, && \text{every relevant internal edge,} \end{aligned}$$

$$J_{load} \models (\mathsf{base} = b \wedge \mathsf{selector} \in D_i), \qquad D_i \text{ finite},$$
$$\mathsf{TargetsFromTable}(b, D_i) = T_{witnessed} = T_{candidate}.$$

The private domain has Top, constants, unsigned intervals and finite word sets. Unknown selector-byte loads may be fresh on every transition; induction does not require equal selector values across loop iterations. The reviewed first scope proves a stable base and dispatch domain, and rejects unsupported relevant writes, opaque calls and unproved clobbers.

Every required captured table entry must already have an independently replayable ordinary witness. The loop rule adds coverage for those candidates; it does not invent loop-only targets. Assertions, region entrances/exits, source hashes and graph epoch are all bound. A new entrance or changed graph invalidates affected receipts and forces revalidation.

This proves a local invariant for arbitrarily many finite traversals within the declared abstraction. It does not prove that the program loop terminates, that all loops are supported, or that this dump followed one of the witnessed ordinary paths.

C Questions and evidence-indexed claims

The implemented typed question is either an alternative-consistency question or a fault-dependency question. An alternative selects an existing typed edge or dispatch and a masked whole-GPR endpoint condition. Fault dependency selects a captured fault access and may focus a source-discovered load or writer. Selectors do not add discovery roots or erase other candidates.

A value fact needs more structure than register name and integer:

$$\begin{aligned}\mathsf{Fact} \simeq \{\, &\mathsf{snapshot} : s,\ \mathsf{activation} : \alpha,\ \mathsf{revision} : e,\\ &\mathsf{point} : (a, \mathsf{Before} + \mathsf{After}),\ \mathsf{location} : \ell,\ \mathsf{width} : w,\\ &\mathsf{instanceScope} : \iota,\ \mathsf{profile} : m,\ \mathsf{premises} : \Pi,\ \mathsf{precision} : \mathsf{Exact} + \mathsf{Constrained} + \mathsf{Unknown}\,\}.\end{aligned}$$

This summarizes the binding requirements across several concrete carriers. Two same-named registers at different points or activations are not interchangeable facts. Exact information is exact only in its declared model, point and instance scope. Compatible refinements can narrow an abstract value; incompatible identities cannot simply be joined into one exact pointer.

The actual question verdict enum is

$$\begin{aligned}\mathsf{QuestionVerdict} = \mathsf{Unknown} + \mathsf{RefutedUnderModel}\\ + \mathsf{PossibleUnderModel} + \mathsf{PremiseConflict}.\end{aligned}$$

A checked witness supports possible-under-model. Refutation needs the relevant coverage and supported semantics. A well-formed hypothesis conflicting with an admitted observation produces premise-conflict; it is not permission to exclude every writer through logical explosion. Malformed proof data remains an error rather than a premise conflict.

Fair bounded scheduling services source-frontier resolution, constraints, scalar propagation and origin collection. Each layer retains its own stop. Source revisions are immutable and bounded; previously checked facts can be reused only when their identities and dependencies remain compatible.

For memory, the base candidate set is preserved. Unknown aliasing, interference and incomplete access/instance coverage prevent blanket writer exclusions. A displayed focus writer changes presentation, not the universe of possible origins. A compatible completed callee may contribute its already validated graph and origins; incomplete caller facts are not promoted to a completed caller analysis.

Alignment hypotheses are also indexed facts. Assuming an effective address is 16-byte aligned admits a selected MOVAPS normal continuation only conditionally. Baseline Unknown remains visible. An unaligned execution of that instruction would take its modeled exception alternative; no actual earlier alignment is observed merely because the investigator selected normal flow.

D Exact compact completion changes representation, not the problem

At the current source revision, caller S6 uses a shared sorted universe $U = (d_0, \ldots, d_{m-1})$ with a membership vector b per row. Its abstraction is

$$\gamma(U, b) = \{d_i \mid 0 \leq i < m \wedge b_i = 1\}.$$

Union, kill masks and generation masks operate exactly on this representation. They do not widen or discard origins. Rust uses 64-bit words; native immediate storage uses 62-bit words internally. The `bitmap/v1` transport has its own canonical dictionary, membership, binding and physical-budget checks; these are representation facts rather than new abstract origins.

Completion-only iterative reverse postorder improves propagation scheduling. Ordinary named-action replay retains its previous schedule. The intended preservation condition is equality of completed states under the same frozen request, not equal action-by-action traces for the optimized route.

`CompletionLimits` default	Bound
Total charged actions	1,000,000
64-bit membership words per plane	33,554,432
Definition identities	1,000,000
Definition-name bytes	64 MiB
Elapsed time	600 s

These are explicit finite policies, not a process-memory guarantee. The solver owns two membership planes. Already-completed prefixes must still satisfy the total-action cap. Bounded refusal does not publish completion. The existing unbounded `finish` API retains its reference fallback if compact workspace policy declines. Native framing and completed-result caps remain separate.

E Separate supplied-stateflow and LLVM IR families

E.1 Finite abstract stateflow

The supplied-stateflow model is parameterized by a finite state catalogue $\mathcal{C}$ and a relation of typed transitions:

$$\mathsf{Step} \subseteq G \times \mathcal{C} \times \mathcal{C}, \qquad X : A \to \mathcal{P}_f(\mathcal{C}).$$

Each abstract state has running/faulted/returned/stopped status and a finite valuation mapping locations to possible abstract values. Entry states, effects, terminal transitions and complete-sites assertions are additional inputs. The stateflow solver propagates supported transitions from admitted entry facts; it does not generate all these semantics from a minidump.

A structural edge may be feasible under a reached transition, provably infeasible under complete supplied source semantics, or of unknown feasibility. The current implementation marks an edge

infeasible only at completed stateflow, when its source is reached and declared complete and the edge has no feasible transition. Unreached sources are tracked separately. *All structural edges remain in the structural graph.*

This family has its own request, validation, native/reference solvers and report schema. Completeness here is completeness of the supplied relation for the declared model, not hardware completeness or evidence that a capture-time register value held at an earlier branch.

E.2 Direct verified LLVM IR

The IR family takes an already supplied and verified function. Its identity records the artifact, module and function. Its node identifiers are instruction and block IDs, not semantic machine VAs. Its normalized structure includes block terminators and successors, SSA value definitions/uses, phi inputs, conservative memory predecessors and visible callee targets.

Writing Z_{IR} for instruction seeds, the dependency closure has the form

$$S^{*}_{IR} = \mathrm{lfp}\left(S \mapsto Z_{IR} \cup S \cup \bigcup_{i \in S} \big(\mathsf{ValueDefPreds}(i) \cup \mathsf{MemoryPreds}(i)\big) \right).$$

The implementation's `dependency_preds` begins with supplied memory predecessors and adds instruction definitions for used SSA values. Phi inputs and block-control facts have their own normalized admission constraints; call-target incompleteness remains an obligation. This is a separate IR dependency analysis. It does not reconstruct original LLVM IR from the dump or silently supply missing binary instruction effects.

The separation matters in a structural type presentation: an $\mathsf{InstructionId}$ or a supplied catalogue state cannot be substituted for $\mathsf{Addr}(s)$ or a captured concrete register merely because their encodings look compatible. The relevant semantic identity and producer contract differ.

F Current evidence and remaining scope

The 2026-10-19 caller S6 record is retained evidence bound to its source capsule, tools, original dump and normalized requests. For the current `184a181` code, its publication receipt confirms that all 980 qualified runtime/model/test/tool/Cargo inputs are unchanged; later publication deltas are documentation-only. This explanation reads that record and current source; it does not rerun the remote qualification.

Retained original profile	Caller frame 1	Callee frame 0
Decoded instruction sites	9,589	778
Edges	11,321	895
Reaching-definition memberships	141,301,903	1,304,602
Charged actions	91,175	3,587

Both saved and raw original requests match, and the completed Rust/native states agree under the declared original local-normal-flow, empty-seed profile. The retained record includes row-by-row independent checks, root/features/ Clippy gates, 113 finite model configurations, 330 replay observations and 29 compiled mutation controls. These are scoped observations, not general release or universal refinement certification.

The original caller-barrier gate is cleared in that profile. Later MOVAPS alignment, two callee indirect jumps, the separate frame-path F4 gate, broader large-report/release qualification, executed history/root cause and universal refinement remain unresolved or separate. Complete caller dataflow can coexist with incomplete control/evidence elsewhere.

G Source map, origin and direct follow-up

Status. Source-grounded explanatory reconstruction of local revision `184a181`; not a new canonical feature design or proof artifact. **Motivation.** Explain the current model through structural types without conflating data shape, admitted evidence and execution history. **Contribution.** The four-level reconstruction, types, judgments and worked example connect concrete carriers to their scoped meanings. **Limits.** Dependent/refinement indices are mathematical notation; conformance and validation remain separate from universal proof.

The direct origins and next readings are the following inspected sources. Links target the local checkout relative to this archived PDF; the manifest records their bytes at the stated revision. A later checkout can change those files, and PDF-reader policy can restrict launching source links:

1. Specs/Ariadne.tla — authoritative normalized core transitions and invariants.
2. Specs/README.md — active model ownership and explicit abstraction boundaries.
3. src/model.rs — actual records, enums and request validator.
4. src/engine.rs — Rust reference state machine and completion entry points.
5. src/completion.rs — bounded exact fixed-point completion.
6. src/definition_set.rs — shared universe and exact membership semantics.
7. src/analysis_view.rs — sealed views and private completed carrier.
8. docs/Ariadne/architecture.md — production ownership and separate IR/stateflow routes.
9. docs/Ariadne/file-reader-design.md — captured-read, discovery and independent entry contract.
10. docs/Ariadne/bap-analysis-core-design.md — native protocol and backend admission boundary.
11. docs/Ariadne/guarded-target-resolution-design.md — candidate/final graph layering and loop-coverage contract.
12. src/bap/target_resolution/loop_closure.rs — actual coverage variants and assertion receipts.
13. docs/Ariadne/investigation-design.md — Stage 5.1, alignment, Stage 5.2 and frame-path distinctions.
14. docs/Ariadne/frame-question-investigation-design.md — question identities, evidence, candidate preservation and fair work.
15. src/investigation/frame_question/model.rs — actual typed questions, verdicts and policies.

16. docs/Ariadne/caller-semantic-coverage-design.md — S6 implementation and its accepted finite profile.
17. evidence/Ariadne/caller-semantic-coverage/20261019/README.md — retained source/tool/corpus-bound qualification, not a new run.
18. docs/Ariadne/semantic-assurance.md — BAP trust decision and open universal refinement boundary.
19. ROADMAP.md — direct successor reading for remaining work and product direction.

For notation, Pfenning's subtyping notes explain subsumption. Harper's *Practical Foundations for Programming Languages*, second-edition preview provides background on judgments, statics and dynamics. The Ariadne-specific formulas in this document are independently reconstructed from the sources above and are not quoted theorems from either text.

The accompanying manifests retain the identity of this explanation:

- `source-manifest.json`: inspected source files and their hashes.
- `render-receipt.json`: PDF, LaTeX source and rendering checks.

These records do not refresh the qualification archives or authenticate a capture.